内部资料，请勿发布到互联网上，违者必究

软件构造实验指导书

IOC 容器设计与实现

课程编号	22WHSE22006	授课教师	辛国栋	适用学期	2025 年秋
适用班级	2311101~04；2311301				

第一章：实验介绍

各位同学大家好。接下来我们计划用四次实验、共16学时完成一个**实验主题：基于注解的IOC容器设计与实现。**

简单介绍一下为什么选择这样一个实验主题。根据《软件学院2023版本科生培养方案》和《22WHSE22006-软件构造》课程大纲的精神，《软件构造》这门课时大平台专业基础必修课。这门课的核心目标是**编写高质量的面向对象代码。**而 Spring 框架是目前企业开发最流行的框架（没有之一），它通过"**控制反转（IoC，Inversion of Control）**"和"**面向切面编程（AOP，Aspect Oriented Programming）**"两个核心机制，帮助我们构建**高内聚、松耦合**的应用程序。在 Spring 框架中，充分考虑了代码的高质量因素：**可维护性，可读性，可扩展性，灵活性，简洁性，可复用性，可测试性**等因素，是高质量代码的集大成者。毫无疑问，将 Spring 框架的源代码作为我们学习的榜样是一个很好的选择，而且完美匹配课程的核心目标。

此外，每年的互联网大厂招聘和研究生面试，有关 Spring 框架的题目必不可少，因此无论是就业还是读研，Spring 框架底层的思想都是我们亟需需要掌握的高端知识。

1. Spring Framework

需要注意的是，Spring 并不等于 Spring Framework。Spring 是指一个非常完整的技术生态体系，包括 Spring Framework，Spring Boot，Spring Cloud ，Spring Batch、Spring Data，Spring Security 等等，俗称 **Spring 全家桶**；而 Spring Framework是 Spring 全家桶中最核心的部分。其他部分都建立在 Spring Framework 的基础上，如图 1 所示。如果你想了解详细 Spring 全家桶的介绍，请参考官网：https://spring.io/

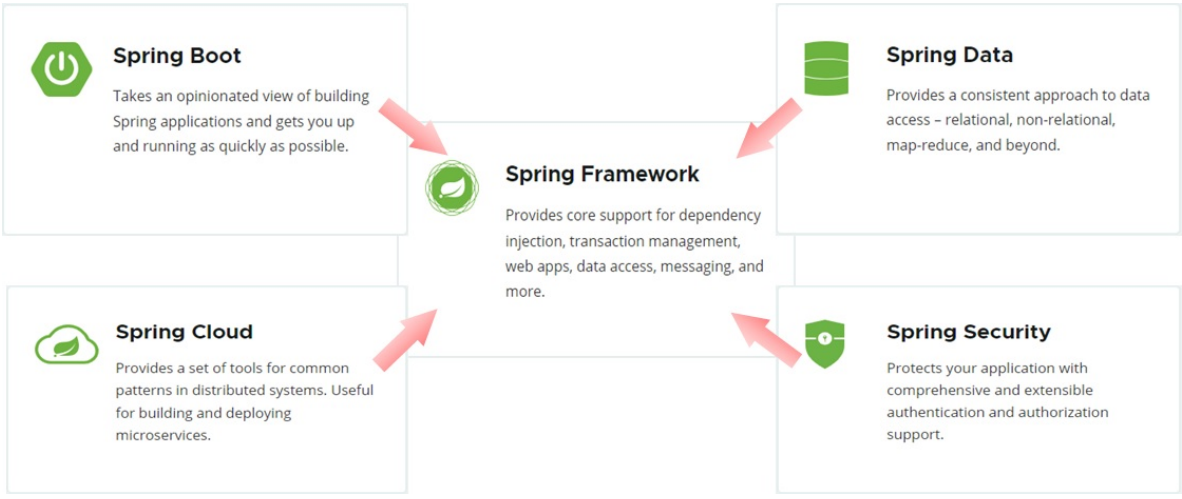


图1：Spring Framework 是 Spring 全家桶的基础

Spring 框架是免费开源的，其代码存放在 GitHub上托管，地址：https://github.com/spring-projects/spring-framework/，从2004年3月发布1.0 版本到目前最新版本 6.2.x，Spring 框架已经变得非常庞大复杂。当然，我们不必通篇去读它的源代码，掌握核心思想足矣。加之课时有限，也没有那么多精力和时间去实现更多的功能。因此，我们的这四次实验，**聚焦在 Spring 框架的 IoC 机制。** Spring 框架的另一个核心机制 **AOP**，不在我们的实验范围之内，感兴趣的同学可以自行实现。

2. 实验内容

在设计实验时，教师已经充分考虑了同学们目前已具备的相关知识及对这些前序知识的掌握程度，知识的难度等问题，采用了"小步快跑"的策略，尽量将知识跨度减小，难度降低。

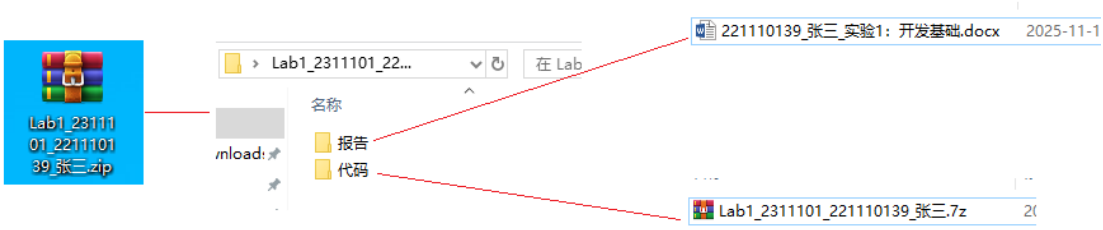
我们按照这样的思路迭代构建四个实验：首先让同学们熟悉注解的获取，并将属性值注入对象；其次将注解扩展到类，将所有的classpath下，将标注@Componet注解的类找到，将属性值解析后进行初始化，最后装入IoC容器。再次实现@Autowired和 @Qualifier，能够将类的属性进行注入；最后，根据已经学过的设计模式，对已经完成的内容进行重构，满足框架的扩展性和灵活性。具体如下：

	内容	目的
实验1：开发基础	完成Java、IDEA快速开发工具的安装和设置；根据基础知识，实验注解和反射的综合应用程序编写；撰写实验报告，并提交纸质版和电子版实验报告	做好开发环境准备，掌握注解和反射的基础知识并能够做基本的应用。
实验2：Bean容器构建	完成注解的代码编写；实现包扫描功能；实现容器的装载；撰写实验报告，并提交纸质版和电子版实验报告	深度应用注解和反射，实现IOC容器的创建和对象装载
实验3：对象的自动装配	重构实验2，实现@Autowired和 @Qualifier，完成Bean对象的自动装配；从容器中获取Bean对象；测试并撰写实验报告，并提交纸质版和电子版实验报告	深度应用注解和反射，实现IOC容器中对象的自动装配
实验4：重构	重构实验3，分离出工厂，能够实现单例、多例；分离出资源类系，为多种形式的资源预留扩展点；测试并撰写实验报告，并提交纸质版和电子版实验报告	应用设计模式；体验可扩展性、可维护性等质量目标如何达成

3. 作业提交

每个实验都需要同学们单独完成，不分组。因此，每个实验都需要提供:

- 纸质实验报告：双面打印。每班学委收齐（收作业的时候，让已经交作业的同学签字，并统计未交同学的姓名学号）。
- 电子版报告及打包后的源代码：新建一个文件夹，命名格式为 “Labx-班号-学号-姓名”，其中Labx中的x，表示实验序号1-4。例如 “Lab1_2311101_221110139_张三” 。在本文件夹下，新建两个子文件夹，分别是 “报告”，“代码”。其中，“报告”实验报告，“代码”存放源代码。存放的文件和代码的压缩包命名格式与顶层文件夹相同，示例如下图：



4. 实验安排

实验时间	地点	内容	班级
13周周五 (2025.11.28) 18:40-19:40	研究院一号楼中519机房	实验1：开发基础	2311101-04
13周周六 (2025.11.29) 14:00-17:40	研究院一号楼中519机房	实验1：开发基础	2311105-08,2311301
14周周五 (2025.12.05) 18:40-19:40	研究院一号楼中519机房	实验2：Bean容器构建	2311101-04
14周周六 (2025.12.06) 14:00-17:40	研究院一号楼中519机房	实验2：Bean容器构建	2311105-08,2311301
15周周五 (2025.12.12) 18:40-19:40	研究院一号楼中519机房	实验3：对象的自动装配	2311101-04
15周周六 (2025.12.13) 14:00-17:40	研究院一号楼中519机房	实验3：对象的自动装配	2311105-08,2311301
16周周五 (2025.12.20) 18:40-19:40	研究院一号楼中519机房	实验4：重构	2311101-04
16周周六 (2025.12.21) 14:00-17:40	研究院一号楼中519机房	实验4：重构	2311105-08,2311301

考虑到部分同学选修了其他课程，如果在本班设置的时间内不能参加，可以在其他班完成实验。纸质和电子资料在完成实验报告一周内提交。

第二章：知识准备

1. 注解 Annotation

注解（Java Annotation） 是 JDK5.0 就引入的一种注释机制，并不是什么新特性。在Java课程的学习过程中，同学们了解最多的注解是“@override"，选修了《web应用开发技术》课程的同学了解的多一点，但基本上处于“**知其然不知其所以然**”的状态。在《软件架构》课程实验中，频繁用到注解 Annotation，因此将 Annotation 这项技术拿出来，重新梳理一下，帮助同学们深入了解和掌握这项技术。如果 Annotation 的知识非常熟悉，可以直接跳过本章。

1.1 注解简介

注解（Annotation），也叫元数据，是一种代码级别的说明。通俗的理解，**就是在代码之上加“@注解名”形式的标记。**它可以声明在包、类、字段、方法、局部变量、方法参数等的前面，用来对这些元素进行说明。注解本身并不是代码的一部分，它们不会直接影响代码的执行，但可以在编译、类加载和运行时被读取和处理。注解为开发者提供了一种以非侵入性的方式为代码提供额外信息的手段。

注解可以和**注释**类比一下。这两者都是对程序进行解释说明，不同的是：注释是给程序员看的，只在 Java 源代码中有效，在编译后的 .class 文件中不存在注释，也就是当源代码编译之后，会进行注释擦除；而**注解**是给虚拟机 JVM 看的。当虚拟机看到注解之后，就知道要做什么事情了。这些事情大致分成三类：包括：

- 编译期检查（如`@Override`确保方法正确重写）
- 生成文档（配合 Javadoc 提取注释信息）
- 运行时处理（如 Spring 通过`@Autowired`实现依赖注入）等。

1.2 JDK的内置注解

从JDK5开始，到目前的 JDK25，Java中有10个内置注解，其中6个注解是作用在代码上的，**4个注解是负责注解其他注解的(即元注解)**，元注解提供对其他注解的类型说明。

注解	作用	作用范围
@Override	检查该方法是否是重写方法。如果其继承的父类或者实现的接口中并没有该方法时，会报编译错误。	作用在代码上
@Deprecated	标记表示过时的，不推荐使用。可以用于修饰方法，属性，类。如果使用被此注解修饰的方法，属性或类，会报编译警告。	作用在代码上
@SuppressWarnings	告诉编译器忽略注解中声明的警告。	作用在代码上
@SafeVarargs	Java 7开始支持，忽略任何使用参数为泛型变量的方法或构造函数调用产生的警告。	作用在代码上
@FunctionalInterface	Java 8开始支持，标识一个匿名函数或函数式接口。	作用在代码上
@Repeatable	Java 8开始支持，标识某注解可以在同一个声明上使用多次。	作用在代码上
@Retention	标识这个注解的保存级别，是只在代码中，还是编入class文件中，或者是在运行时可以通过反射访问。包含关系runtime>class>source。	作用在其他注解上，即元注解
@Documented	标记这些注解是否包含在用户文档中javadoc。	作用在其他注解上，即元注解
@Target	标记某个注解的使用范围，例如作用方法上，类上，属性上等等。如果注解未使用@Target，则注解可以用于任何元素上。	作用在其他注解上，即元注解
@Inherited	说明子类可以继承父类中的此注解，但这不是真的继承，而是可以让子类Class对象使用getAnnotations()获取父类被@Inherited修饰的注解	作用在其他注解上，即元注解

以@Override，@Deprecated，@SuppressWarnings为例，简单说明 Java 内置注解的使用方法。元注解的使用在自定义注解部分进行解释。

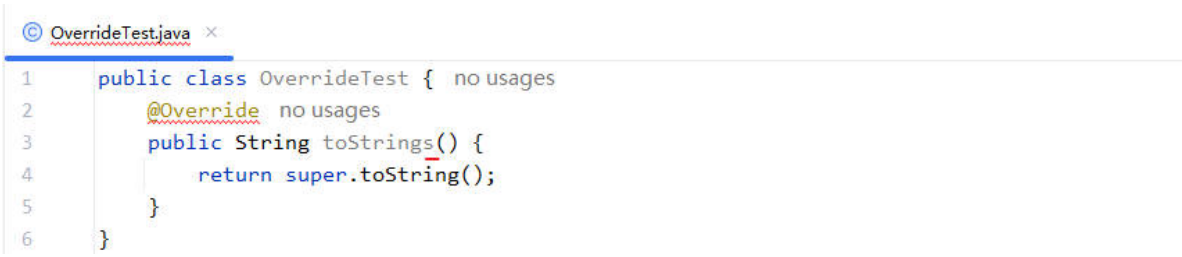
1.2.1 @Override注解

> 标记在成员方法上，用于标识当前方法是重写父类（父接口）方法，编译器在对该方法进行编译时会检查是否符合重写规则，如果不符合，编译报错

在Java中，如果类没有明确写明继承的父类，那么它的父类就是Object。 Object 类中有一个方法是toString方法，通常在实体类中去重写此方法来达到打印对象信息的效果，这时候也会发现重写的toString方法上方就有一个@Override注解。如下所示：

```
public class OverrideTest {
    @Override
    public String toString() {
        return super.toString();
    }
}
```

当试图去改变重写后的toString方法名称，将方法名改为toString**s**。会发现在编译期就报错，如下所示：



说明toStrings() 方法并不是覆盖父类（Object）的方法。这就体现了@Override注解的作用：**检查是否符合重写规则，如果不符合，编译报错**

1.2.2 @Deprecated注解

> 用于标记当前类、成员变量、成员方法或者构造方法过时如果开发者调用了被标记为过时的方法，编译器在编译期进行警告。

某类 DeprecatedTest（不妨将具体化为MySql的驱动程序），假设当前是V1.0版本，它为用户提供了一个接口，在类中表现就是show1方法。

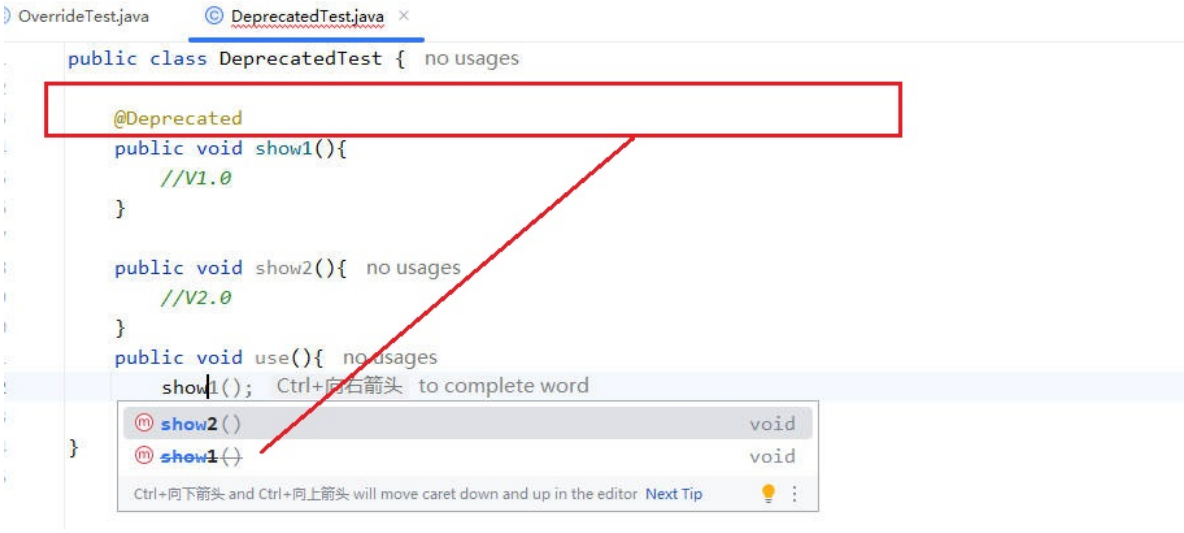
```
public class DeprecatedTest {
    public void show1(){
        //V1.0
    }
}
```

随着时间推移，发现show1方法提供的功能不能满足新的情况，这时需要对show1方法进行更新，类变成V2.0。这就带来了两个问题：

- 问题1：是不是直接在show1方法上改功能？答案是否定的。如果直接在show1方法上改，之前使用该接口的客户，他们的程序都会失效。所以，应该保留show1方法，继续为老版本的用户服务，对更新后的版本，新开一个方法show2

```
public class DeprecatedTest {
    public void show1(){
        //V1.0
    }
    public void show2(){
        //V2.0
    }
}
```

- 问题2：当用户在后续开发的时候，如何知道使用哪个版本的show方法，是show1，还是show2？需要在show1方法上方加@Deprecated注解给予提示。通过这种方式也告知用户“这是旧版本时候的功能，不建议再继续使用旧版本的功能”。如下图所示，用户在使用的过程中就会发现，方法中的提示是在调用的方法名上加了一道删除线。这就体现了show1方法过时了，已经不建议使用。

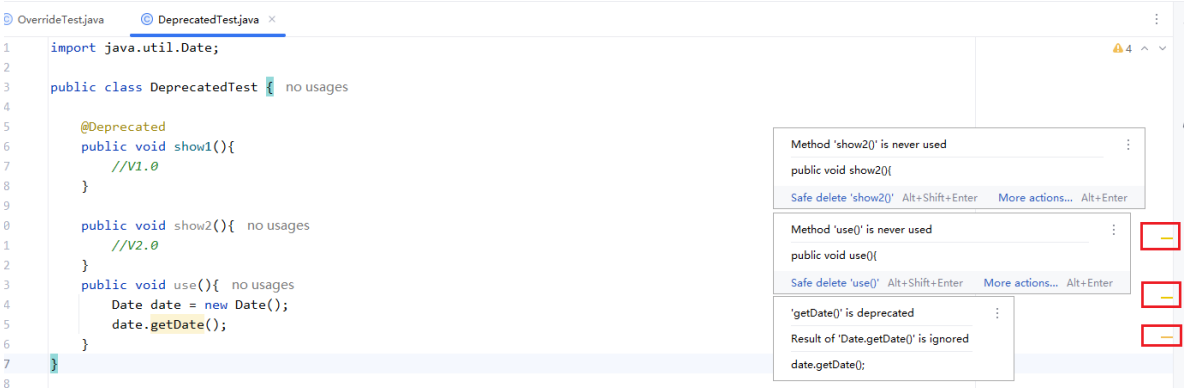


这也就是@Deprecated注解的作用

1.2.3 @SuppressWarnings注解

> 压制警告注解，可放置在类和方法上，该注解的作用是阻止编译器发出某些警告信息，该注解为单值注解，只有 一个value参数，该参数为字符串数组类型，参数值常用的有如下几个。
>
> - unchecked：未检查的转化，如集合没有指定类型还添加元素
> - unused：未使用的变量
> - resource：有泛型未指定类型
> - path：在类路径，原文件路径中有不存在的路径
> - deprecation：使用了某些不赞成使用的类和方法
> - fallthrough：switch语句执行到底没有break关键字
> - rawtypes：没有写泛型，比如: List list = new ArrayList();
> - all：全部类型的警告

压制警告注解，顾名思义就是压制警告的出现。在Java代码的编写过程中，Idea的代码编辑器的右侧是有很多黄色警告出现，如下图所示：



如果不想让这些警告信息出现，就可以在类上，使用@SuppressWarnings("all") 注解来压制所有的警告信息，如下图所示

```java
import java.util.Date;

@SuppressWarnings("all")  no usages
public class DeprecatedTest {

    @Deprecated
    public void show1(){
        //V1.0
    }

    public void show2(){  no usages
        //V2.0
    }
    public void use(){  no usages
        Date date = new Date();
        date.getDate();
    }
}
```

@SafeVarargs，@FunctionalInterface ，@Repeatable这三个注解，大家仔细查阅文档，不再赘述。

1.3 自定义注解

1.3.1 注解的本质

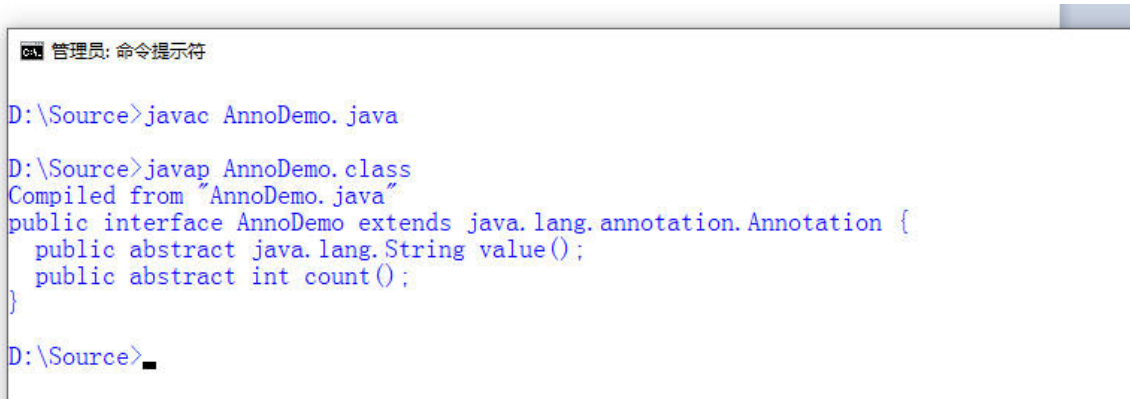
使用@interface关键字自定义注解，其实底层就是定义了一个接口，而且自动继承`java.lang.annotation.Annotation`接口。例如：我们自定义一个注解如下：

```java
public @interface MyAnnotation {
    String value() default "";
    int count();
}
```

其中：

```java
    String value() default "";
    int count();
```

表示注解的属性。此外，我们也不知道@Interface是什么。可以通过一个简单的方式来验证：那就把自定义的这个注解反编译一下，看一下反编译信息。反编译操作如下：



反编译的内容：

```java
public interface AnnoDemo extends java.lang.annotation.Annotation {
  public abstract java.lang.String value();
  public abstract int count();
}
```

- 首先，看过反编译内容后，可以直观的得知它是一个接口，因为它的 public 修饰符后面的关键字是 interface。
- 其次，我们发现MyAnnotation这个接口是继承了`java.lang.annotation`包下的`Annotation`接口。

- 再次，其属性，不管叫做属性，还是参数，还是方法，本质上确实是抽象方法。

所以，我们可以得知注解的本质就是一个接口，该接口默认继承了`Annotation`接口。

注解的定义内容如下：

- 格式为public @interface 注解名
- 内部的每一个方法实际是声明了一个参数，方法的名称就是参数的名称。
- 返回值类型就是参数的类型，而且返回值类型只能是基本类型（int，float，long，short，boolean，byte，double，char），Class，String，enum，Annotation以及上述类型的数组形式。
- 如果定义了参数，可通过default关键字声明参数的默认值（默认值应该和返回类型匹配），若不指定默认值，使用时就一定要显示赋值，而且不允许使用null值，一般会使用空字符串或者0。
- 如果只有一个参数，一般参数名为value，因为使用注解时，赋值可以不显示写出参数名，直接写参数值。

```
import java.lang.annotation.*;

@Target(ElementType.METHOD) // 此注解只能用在方法上。
@Retention(RetentionPolicy.RUNTIME) // 此注解保存在运行时期，可以通过反射访问。
@Inherited // 说明子类可以继承此类的此注解。
@Documented // 此注解包含在用户文档中。
public @interface CustomAnnotation {
    String value(); // 使用时需要显示赋值，因为该属性没有设置默认值
    int id() default 0; // 有默认值，使用时可以不赋值
}
```

1.3.2 @Target 元注解

查看 Target 注解的源码 , 该注解只有 1 11 个注解属性 , 且属性值是 value , 因此在给 注解属性 赋值时 , 可以省略 注解属性名称 ;

```
@Documented
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.ANNOTATION_TYPE)
public @interface Target {
    ElementType[] value();
}
```

value 注解属性 的类型是 ElementType[] 数组类型 , 代表 注解 作用的位置 ; 可以设置的位置参考 ElementType 枚举类 ;

- ElementType.TYPE : 注解作用于类、接口、枚举 ;
- ElementType.FIELD: 注解作用于字段上 ;
- ElementType.METHOD : 注解作用于方法上 ;
- ==========上面的三个常用，下面的大家有个印象即可==========
- ElementType.ANNOTATION: 注解作用于注解上，即给注解进行注解 ;
- ElementType.CONSTRUCTOR: 注解作用于构造方法上 ;
- ElementType.LOCAL_VARIABLE: 注解作用于局部变量上 ;
- ElementType.PACKAGE: 注解作用于包上 ;
- ElementType.TYPE_PARAMETER: 可以用来表示`T`类型参数的声明

@Target 使用示例 : 使用 `@Target(ElementType.TYPE)` , 根据ElementType.TYPE，可以知道，设置注解作用位置必须是 类/接口/枚举 , 将 @Annotation 标注在其它位置会报错 ;

例如 1.3.1 中我们的自定义注解CustomAnnotation，为了方便阅读，将其再：其Target设定为ElementType.METHOD，表示注解作用于方法上，如果标注在其他地方就会报错：

```
public class TestAnnotation {

    // @CustomAnnotation(value = "test") 只能注解在方法上，这里会报错
    private String str = "Hello World!";

    @CustomAnnotation(value = "test")
    public static void main(String[] args) {
        System.out.println(str);
    }
}
```

Target 注解中 , ElementType[] value() 注解属性的类型是数组类型 , 说明可以设置多个注解标注位置 , 例如下列代码说明 某注解可以标注到类，属性和方法上。

```
@Target({ElementType.TYPE, ElementType.FIELD, ElementType.METHOD})
```

1.3.3 @Retention 元注解

@Retention 注解用于说明该注解需要保留到什么阶段 ;

Java 代码的三个阶段分别是 : 源代码阶段 , 类对象阶段 , 运行时阶段 ;

查看 Retention 源码 :

```
@Documented
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.ANNOTATION_TYPE)
public @interface Retention {
    RetentionPolicy value();
}
```

Retention 注解的 注解属性 `RetentionPolicy value()` , 其类型是 RetentionPolicy 枚举类型 , 三个枚举值 , 分别对应 Java 代码三大阶段 源代码阶段 , 类对象阶段 , 运行时阶段 ， 查看JDK的源代码，可以很清楚了解每个枚举值的用处，具体如下：

```
public enum RetentionPolicy {
    /**
     * 源代码阶段
     * 编译器丢弃该注解.
     */
    SOURCE,
    /**
     * 类对象阶段
     * 注解会被保存到字节码文件中, 不会被 JVM 读取到.
     */
    CLASS,
    /**
     * 运行时阶段
     * 注解会被保存到字节码文件中, 并会被 JVM 读取到.
```

```
     *
     * @see java.lang.reflect.AnnotatedElement
     */
    RUNTIME
}
```

一般情况下，开发则自定义注解，都是在运行时进行一些代码分析，设置 RetentionPolicy.RUNTIME 注解属性；

```
@Retention(RetentionPolicy.RUNTIME)
```

1.3.4 @Documented 元注解

如果添加了该注解，表示当前注解会被抽取到 Java API 文档中；

```
@Documented
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.ANNOTATION_TYPE)
public @interface Documented {
}
```

为了让大家更清晰了解，我们对1.3.1 中的自定义注解CustomAnnotation进行修改，添加可以在类上标注

```
@Target({ElementType.METHOD,ElementType.TYPE}) // 此注解用在方法和类上。
@Retention(RetentionPolicy.RUNTIME) // 此注解保存在运行时期，可以通过反射访问。
@Inherited // 说明子类可以继承此类的此注解。
@Documented // 此注解包含在用户文档中。
public @interface CustomAnnotation {
    String value(); // 使用时需要显示赋值
    int id() default 0; // 有默认值，使用时可以不赋值
}
```

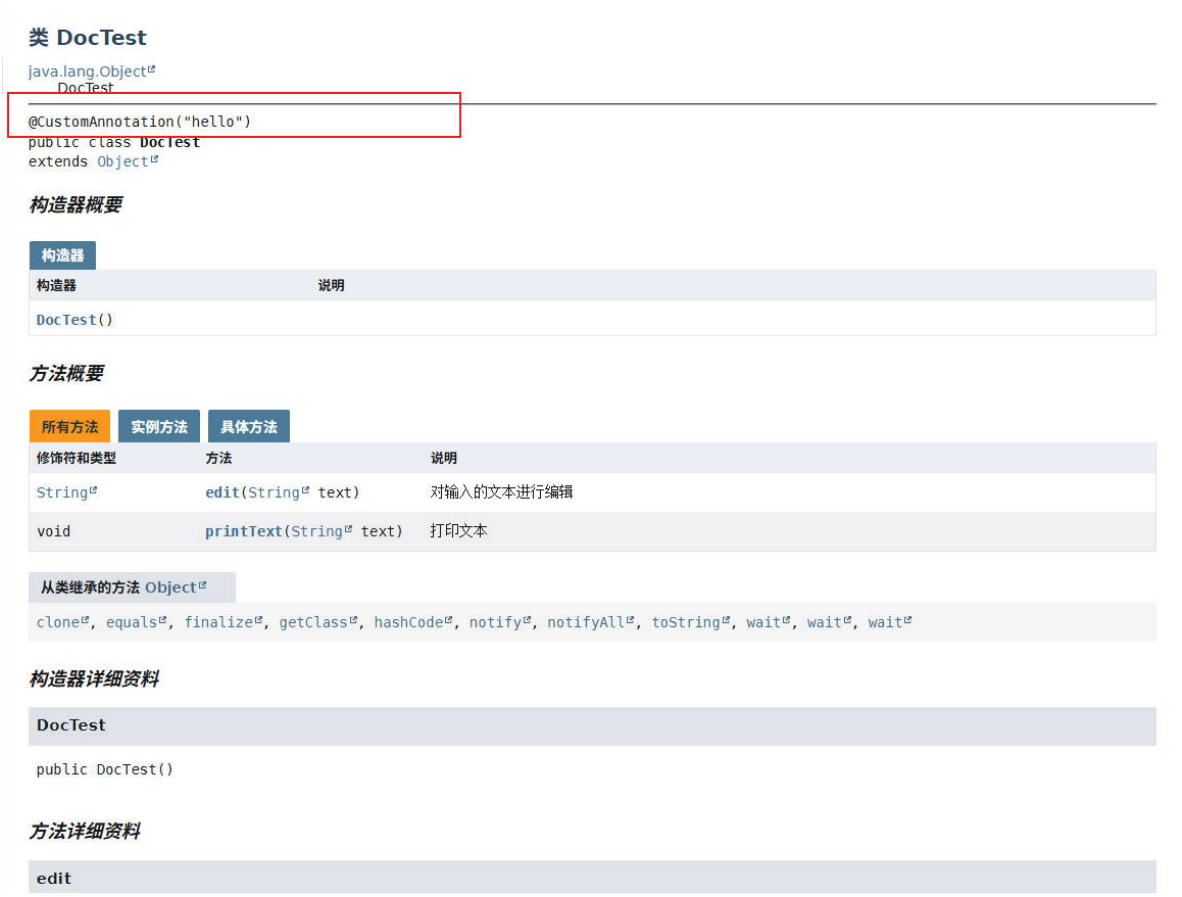
接下来我们用CustomAnnotation注解标注我们自定义类：

```
@CustomAnnotation(value = "hello") // value没有设置默认值，必须显式赋值
public class DocTest {

    /**
     * 对输入的文本进行编辑
     * @param text 输入的文本
     * @return 编辑后的文本
     */
    public String edit(String text) {
        return text;
    }

    /**
     * 打印文本
     * @param text 待打印的文本
     */
    public void printText(String text) {
        System.out.println(text);
    }
}
```

使用 `javadoc DocTest.java` 命令 , 生成 Java 文档 ; 如果使用 @Documented 注解修饰 CustomAnnotation注解 , 凡是用CustomAnnotation注解的类，生成的文档都会在注解的位置显示相应注解 ;



类 DocTest

java.lang.Object
DocTest

```
@CustomAnnotation("hello")
public class DocTest
extends Object
```

构造器概要

构造器	说明
DocTest()	

方法概要

修饰符和类型	方法	说明
String	edit(String text)	对输入的文本进行编辑
void	printText(String text)	打印文本

从类继承的方法 Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

构造器详细资料

DocTest

```
public DocTest()
```

方法详细资料

edit

例如，在DocTest的方法上，加一下CustomAnnotation注解，我们看一下生成的文档截图：



printText

```
@CustomAnnotation("printText")
public void printText(String text)
```

打印文本

参数:

text - 待打印的文本

1.3.5 @Inherited

@Inherited : 描述 注解 是否被子类继承 , 如果添加了该注解 , 则子类会自动继承父类的注解 ;

例如：使用 @Inherited 元注解 标注 CustomAnnotation注解。

使用 CustomAnnotation 注解 标注父类 Person ;子类 Student 继承 Person 类 , 子类中不添加注解 , 那么父类 Person 中的注解自动添加给子类 Student ;

可以通过反射的方式来验证子类是否存在从父类的继承下来的注解。

2. Java反射机制 Reflection

2.1 什么是反射 Reflection

我们先了解下什么是静态语言和动态语言。动态语言是指在运行时可以改变其自身结构的语言。简单说即是在运行时代码可以根据某些条件改变自身结构。动态语言主要有C#，Object-C，JavaScript，PHP，Python等。静态语言是指运行时结构不可改变的语言，例如Java，C，C++等。

Java不是动态语言，但是它可以利用反射机制获得类似动态语言的特性。

反射机制允许程序在执行期取得任何类的内部信息，并能直接操作任意对象的内部属性以及方法等。类在被加载完之后，会在堆内存的方法区中生成一个Class类型的对象，一个类只有一个Class对象，这个对象包含了类的结构信息。我们可以通过这个对象看到类的结构。

我们首先编写一个Java类Student

```
package edu.hitwh.reflect;

public class Student {
    private String name;
    private int age;
    public String gender;
    public String address;

    public Student() {
    }

    public Student(String name) {
        this.name = name;
    }

    private Student(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public Student(String name, int age, String address) {
        this.name = name;
        this.age = age;
        this.address = address;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public int getAge() {
        return age;
    }

    public void setAge(int age) {
        this.age = age;
    }

    public String toString() {
        return "Student{name = " + name + ", age = " + age + "}";
    }
```

```
    public void study() {
        System.out.println("学生在学习");
    }

    private void sleep() {
        System.out.println("学生在睡觉");
    }

    public String eat(String something) {
        System.out.println("学生在吃" + something);
        return "学生已经吃完了，非常happy";
    }
}
```

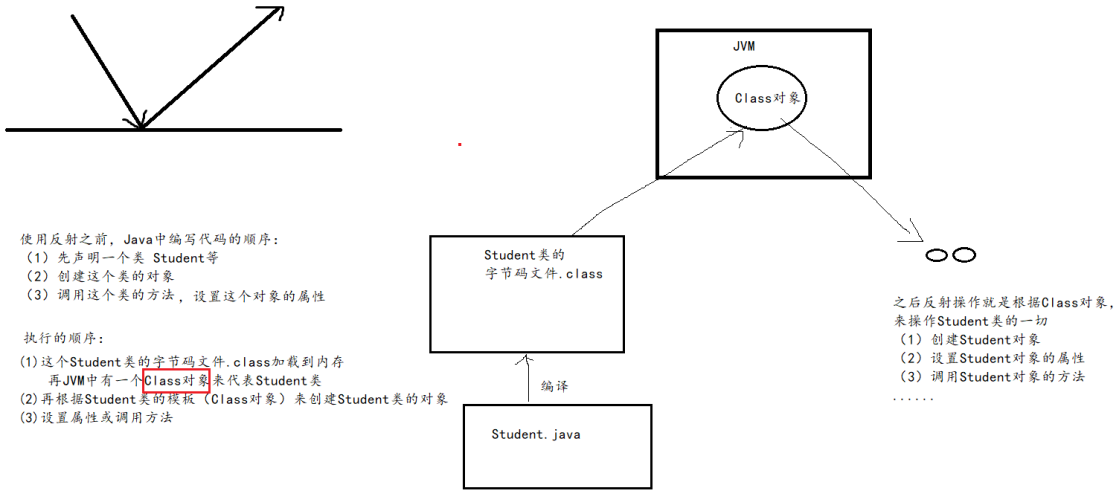
用大家更容易理解的语言，可以这样理解：

Java文件：就是我们自己编写的java代码。例如 Student 类的源代码 Student.java

字节码文件：就是通过java文件编译之后的.class文件（是在硬盘上真实存在的，用眼睛能看到的）,例如 Student.class

字节码文件对象：当.class文件加载到内存之后，虚拟机就会自动创建出来的Class类型的对象。这个对象里面至少包含了：构造方法，成员变量，成员方法。而我们的反射获取的就是字节码文件对象，这个对象在内存中是唯一的。

具体的过程，如下图所示：



显然，我们使用反射，主要是掌握以下几个方面：

- 如何获取class字节码文件的对象
- 利用反射如何获取构造方法（有了构造方法，我们就可以创建对象）
- 利用反射如何获取成员变量（拿到成员变量就可以赋值，获取值）
- 利用反射如何获取成员方法（有了成员方法，就可以运行该成员方法）

2.2 获取字节码文件对象的三种方式

- Class.forName(String 全限定名)： 使用Class类中的静态方法 获取Class对象（最常用）
- 使用类名.class
- 使用Object类中的getClass() 方法 获取到Class对象

代码示例：

```
//1.Class这个类里面的静态方法forName
```

```
//Class.forName("类的全类名"):  全类名 = 包名 + 类名
Class clazz1 = Class.forName("edu.hitwh.reflect.Student");
//源代码阶段获取 --- 先把Student加载到内存中，再获取字节码文件的对象
//clazz1 就表示Student这个类的字节码文件对象。
//就是当Student.class这个文件加载到内存之后，产生的字节码文件对象

//2.通过class属性获取
//类名.class
Class clazz2 = Student.class;
//因为class文件在硬盘中是唯一的，所以，当这个文件加载到内存之后产生的对象也是唯一的
System.out.println(clazz1 == clazz2);//true

//3.通过Student对象获取字节码文件对象
Student s = new Student();
Class clazz3 = s.getClass();
System.out.println(clazz1 == clazz2);//true
System.out.println(clazz2 == clazz3);//true
```

2.3 获取构造方法

API的规则：get表示获取， Declared表示包含私有， 最后的s表示所有，复数形式

如果当前获取到的是私有的，必须要临时修改访问权限，否则无法使用

方法名	说明
Constructor<?>[] getConstructors()	获得所有的构造（只能public修饰）
Constructor<?>[] getDeclaredConstructors()	获得所有的构造（包含private修饰）
Constructor getConstructor(Class<?>... parameterTypes)	获取指定构造（只能public修饰）
Constructor getDeclaredConstructor(Class<?>... parameterTypes)	获取指定构造（包含private修饰）

代码示例：

```
package edu.hitwh.reflect;

import java.lang.reflect.Constructor;

public class ReflectDemo1 {
    public static void main(String[] args) throws ClassNotFoundException,
NoSuchMethodException {
        //1.获得整体（class字节码文件对象）
        Class clazz = Class.forName("edu.hitwh.reflect.Student");
        //2.获取构造方法对象
        //获取所有构造方法（public）
        Constructor[] constructors1 = clazz.getConstructors();
        for (Constructor constructor : constructors1) {
            System.out.println(constructor);
        }
        System.out.println("======================");
```

```
        //获取所有构造（带私有的）
        Constructor[] constructors2 = clazz.getDeclaredConstructors();

        for (Constructor constructor : constructors2) {
            System.out.println(constructor);
        }
        System.out.println("======================");

        //获取指定的空参构造
        Constructor con1 = clazz.getConstructor();
        System.out.println(con1);
        //获取指定的有参构造
        //Constructor con2 = clazz.getConstructor(String.class,int.class); // 报
错，getConstructor只能获得public型的构造
        //System.out.println(con2);

        Constructor con2 = clazz.getDeclaredConstructor(String.class,int.class);
        System.out.println(con2);

        System.out.println("======================");
        //获取指定的构造(所有构造都可以获取到，包括public包括private)
        Constructor con3 = clazz.getDeclaredConstructor();
        System.out.println(con3);
        //了解 System.out.println(con3 == con1);
        //每一次获取构造方法对象的时候，都会新new一个。

        Constructor con4 = clazz.getDeclaredConstructor(String.class);
        System.out.println(con4);
    }
}
```

2.4 获取构造方法并创建对象

涉及到的方法：newInstance

代码示例：

```
//测试类中的代码：
//需求1：
//获取空参，并创建对象

//1.获取整体的字节码文件对象
Class clazz = Class.forName("edu.hitwh.reflect.Student");
//2.获取空参的构造方法
Constructor con = clazz.getConstructor();
//3.利用空参构造方法创建对象
Student stu = (Student) con.newInstance();
System.out.println(stu);
System.out.println("=========================================");

//测试类中的代码：
//需求2：
//获取带参构造，并创建对象
//1.获取整体的字节码文件对象
Class clazz = Class.forName("edu.hitwh.reflect.Student");
//2.获取有参构造方法
Constructor con = clazz.getDeclaredConstructor(String.class, int.class);
```

```
//3.临时修改构造方法的访问权限（暴力反射）
con.setAccessible(true);
//4.直接创建对象
Student stu = (Student) con.newInstance("zhangsan", 23);
System.out.println(stu);
```

2.5 获取成员变量

成员变量的类是 Field。相关api：

API的规则：get表示获取， Declared表示包含私有， 最后的s表示所有，复数形式

如果当前获取到的是私有的，必须要临时修改访问权限，否则无法使用

方法名：

方法名	说明
Field[] getFields()	返回所有成员变量对象的数组（只能拿public的）
Field[] getDeclaredFields()	返回所有成员变量对象的数组，存在就能拿到
Field getField(String name)	返回单个成员变量对象（只能拿public的）
Field getDeclaredField(String name)	返回单个成员变量对象，存在就能拿到

代码示例：

```
public class ReflectDemo2 {
    public static void main(String[] args) throws ClassNotFoundException,
NoSuchFieldException {
        //获取成员变量对象
        //1.获取class对象
        Class clazz = Class.forName("edu.hitwh.reflect.Student");

        //2.获取成员变量的对象（Field对象)只能获取public修饰的
        Field[] fields1 = clazz.getFields();
        for (Field field : fields1) {
            System.out.println(field);
        }

        System.out.println("==============================");

        //获取成员变量的对象（public + private）
        Field[] fields2 = clazz.getDeclaredFields();
        for (Field field : fields2) {
            System.out.println(field);
        }

        System.out.println("==============================");
        //获得单个成员变量对象
        //如果获取的属性是不存在的，那么会报异常
        //Field field3 = clazz.getField("aaa");
        //System.out.println(field3);//NoSuchFieldException

        Field field4 = clazz.getField("gender");
        System.out.println(field4);
```

```
        System.out.println("==============================");
        //获取单个成员变量（私有）
        Field field5 = clazz.getDeclaredField("name");
        System.out.println(field5);
    }
}
```

2.6 获取成员变量并获取值和修改值

方法	说明
void set(Object obj, Object value)	赋值
Object get(Object obj)	获取值

代码示例：

```
public class ReflectDemo3 {
    public static void main(String[] args) throws ClassNotFoundException,
NoSuchFieldException, IllegalAccessException {
        Student s = new Student("zhangsan",23,"广州");
        Student ss = new Student("lisi",24,"北京");
        //需求：
        //利用反射获取成员变量并获取值和修改值

        //1.获取class对象
        Class clazz = Class.forName("edu.hitwh.reflect.Student");

        //2.获取name成员变量
        //field就表示name这个属性的对象
        Field field = clazz.getDeclaredField("name");
        //临时修饰他的访问权限
        field.setAccessible(true);

        //3.设置(修改)name的值
        //参数一：表示要修改哪个对象的name？
        //参数二：表示要修改为多少？
        field.set(s,"wangwu");

        //3.获取name的值
        //表示我要获取这个对象的name的值
        String result = (String)field.get(s);

        //4.打印结果
        System.out.println(result);
        System.out.println(s);
        System.out.println(ss);
    }
}
```

2.7 获取成员方法

成员方法的类是 Method。

API规则： get表示获取； Declared表示私有； 最后的s表示所有，复数形式

如果当前获取到的是私有的，必须要临时修改访问权限，否则无法使用

方法名	说明
Method[] getMethods()	返回所有成员方法对象的数组（只能拿public的）
Method[] getDeclaredMethods()	返回所有成员方法对象的数组，存在就能拿到
Method getMethod(String name, Class<?>... parameterTypes)	返回单个成员方法对象（只能拿public的）
Method getDeclaredMethod(String name, Class<?>... parameterTypes)	返回单个成员方法对象，存在就能拿到

测试代码示例：

```
package edu.hitwh.reflect;

import java.lang.reflect.Method;

public class ReflectDemo4 {
    public static void main(String[] args) throws ClassNotFoundException,
NoSuchMethodException {
        //1.获取class对象
        Class<?> clazz = Class.forName("edu.hitwh.reflect.Student");

        //2.获取方法
        //getMethods可以获取父类中public修饰的方法
        Method[] methods1 = clazz.getMethods();
        for (Method method : methods1) {
            System.out.println(method);
        }

        System.out.println("============================");
        //获取所有的方法（包含私有）
        //但是只能获取自己类中的方法
        Method[] methods2 = clazz.getDeclaredMethods();
        for (Method method : methods2) {
            System.out.println(method);
        }

        System.out.println("============================");
        //获取指定的方法（空参）
        Method method3 = clazz.getMethod("study");
        System.out.println(method3);

        // 获取指定的方法,eat方法带参数，需要将参数类型作为入参传入，例如String.class
        Method method4 = clazz.getMethod("eat", String.class);
        System.out.println(method4);

        //获取指定的私有方法
        Method method5 = clazz.getDeclaredMethod("sleep");
        System.out.println(method5);
    }
}
```

2.8 获取成员方法并运行

方法

Object invoke(Object obj, Object... args)：运行方法

参数一：用obj对象调用该方法

参数二：调用方法的传递的参数（如果没有就不写）

返回值：方法的返回值（如果没有就不写）

代码示例：

```
package edu.hitwh.reflect;

import java.lang.reflect.InvocationTargetException;
import java.lang.reflect.Method;

public class ReflectDemo5 {
    public static void main(String[] args) throws ClassNotFoundException,
NoSuchMethodException, InvocationTargetException, IllegalAccessException {
        //1.获取字节码文件对象
        Class clazz = Class.forName("edu.hitwh.reflect.Student");

        //2.获取一个对象
        //需要用这个对象去调用方法
        Student s = new Student();

        //3.获取一个指定的方法
        //参数一：方法名
        //参数二：参数列表，如果没有可以不写，有的话，需要按照顺序将方法的参数类型作为入参写入
        Method eatMethod = clazz.getMethod("eat", String.class);

        //运行
        //参数一：表示方法的调用对象
        //参数二：方法在运行时需要的实际参数
        //注意点：如果方法有返回值，那么需要接收invoke的结果
        //如果方法没有返回值，则不需要接收
        String result = (String) eatMethod.invoke(s, "重庆小面");
        System.out.println(result);
    }
}
```

3. 注解和反射的综合应用

本章实现自定义简单的单元测试框架。

3.1 需求描述及分析

假设某待测试类为：

```
package edu.hitwh.reflect.practice;

/**
 * 待测试类，自定义的计算器
```

```
 */
public class Calculator {
    public int add(int a, int b) {
        System.out.println(a + " + " + b + " = " + (a + b));
        return a + b;
    }

    public int subtract(int a, int b) {
        System.out.println(a + " - " + b + " = " + (a - b));
        return a - b;
    }

    public int multiply(int a, int b) {
        System.out.println(a + " * " + b + " = " + (a * b));
        return a * b;
    }

    public int divide(int a, int b) {
        System.out.println(a + " / " + b + " = " + (a / b));
        return a / b;
    }
}
```

对应该Calculator类的测试类为：

```
package edu.hitwh.reflect.practice;

public class CalculatorTest {
    Calculator calc = new Calculator();

    @Check
    public void testAdd() {
        calc.add(1,2);
    }
    @Check
    public void testSubtract() {
        calc.subtract(1,2);
    }
    @Check
    public void testMultiply() {
        calc.multiply(1,2);
    }
    @Check
    public void testDivide() {
        calc.divide(1,2);
    }
}
```

请编写一个测试框架，能够运行 CalculatorTest，实现所有有@Check 注解的方法能够自动运行。

思路分析：

- 第1步：先获取到CalculatorTest类对象 即Class对象
- 第2步：根据Class对象获取到CalculatorTest类中所有的方法
- 第3步：根据方法对象获取到方法上方添加的注解对象
- 第4步：根据注解对象，判断是否是“@Check”，如果是，那么获取方法并运行。测试类中的方法都是无入参，返回值为void的。这是测试框架的约定。

相应的API如下：

Object invoke(Object obj, Object... args)：运行方法

- 参数一：用obj对象调用该方法
- 参数二：调用方法的传递的参数（如果没有就不写）
- 返回值：方法的返回值（如果没有就不写）

3.2 定义注解

当前@Check注解并不存在，因此需要定义该注解

```
package edu.hitwh.reflect.practice;

import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Target(ElementType.METHOD)  // 标注在方法上
@Retention(RetentionPolicy.RUNTIME) // 运行时阶段:注解会被保存到字节码文件中，并会被
JVM 读取到.
public @interface Check {
    // 不需要属性
}
```

3.3 定义测试框架

```
package edu.hitwh.reflect.practice;

import java.lang.reflect.InvocationTargetException;
import java.lang.reflect.Method;

public class CheckFramewok {
    public static void main(String[] args) throws ClassNotFoundException,
InstantiationException, IllegalAccessException, InvocationTargetException {
        // 第1步：先获取到CalculatorTest类对象 即Class对象
        Class<?> checkTest =
Class.forName("edu.hitwh.reflect.practice.CalculatorTest");
        //第2步：根据Class对象获取到CalculatorTest类中所有的方法
        Method[] declaredMethods = checkTest.getDeclaredMethods();
        //第3步：根据方法对象获取到方法上方添加的注解对象
        Object object = checkTest.newInstance();
        for (Method method : declaredMethods) {
            if (method.isAnnotationPresent(Check.class)) {
                method.invoke(object);
            }
        }
    }
}
```